

AstraQuantum Tech

Summer of Cybersecurity 2026

Week 6 - Practical Assignment

Cloud Security & Professional Security Assessment

Cloud Security Assessment Report

Student Name	AbdullahZubair
Student ID / Roll No.	AQT-1203
Report Date	20 September 2026
Engagement Type	Authorised training-lab assessment (Pwned Labs, flaws.cloud, TryHackMe)
Classification	Confidential - Academic Submission

1. Executive Summary

This report documents the cloud-security training completed for Week 7 of the AstraQuantum Tech Summer of Cybersecurity 2026 programme. All activity was performed exclusively within purpose-built, explicitly authorised training environments (Pwned Labs, flaws.cloud and TryHackMe). No system outside those environments was scanned, accessed or tested.

Three activities were completed in full, covering the two highest-risk areas of Amazon Web Services (AWS) - identity and storage:

- Intro to AWS IAM Enumeration (Pwned Labs) - IAM-focused lab, completed.
- flaws.cloud Level 1 - storage / cloud-risk lab, completed.
- AWS Shared Responsibility Model (TryHackMe) - cloud-security room, completed 100%.

Across the practical labs I carried out identity enumeration, resource enumeration and authorised data access against intentionally vulnerable targets. Two findings are documented in Section 6, the more serious being an over-permissioned IAM user whose exposed keys allowed access to a sensitive internal S3 bucket (CLOUD-01, High), and a publicly listable S3 bucket exposing a hidden file (CLOUD-02, Medium). In both cases the weakness lay on the customer side of the AWS Shared Responsibility Model. Four evidence screenshots are provided in Appendix A.

2. Scope and Rules of Engagement

2.1 In Scope

Only the intentionally vulnerable lab environments provided or approved by the assignment: the Pwned Labs "Intro to AWS IAM Enumeration" lab (IAM user dev01 in AWS account 794929857501), the flaws.cloud training environment (bucket flaws.cloud, region us-west-2), and the TryHackMe "AWS Shared Responsibility Model" room. All targets were reachable solely through my own authenticated account or via anonymous access explicitly permitted by the lab.

2.2 Out of Scope

- Any AWS account, bucket, host or application not provisioned by the assigned labs.
- Other users of the training platforms and the platform infrastructure itself.
- Destructive actions, denial-of-service testing, and any data exfiltration beyond what a guided lab step instructs.
- Social engineering and physical security testing.

2.3 Rules of Engagement

- Testing was performed solely for education, in environments designed and licensed for that purpose.
- No credentials, flags or findings from these labs are reused against any real system.

- All evidence was captured from my own workstation and my own accounts, with the system date and time visible.
- Activity remained within the boundaries of each lab; no attempt was made to pivot outside the assigned target.

Authorisation basis: the Pwned Labs, flaws.cloud and TryHackMe environments are provided expressly for hands-on practice under each platform's terms of service, which constitutes the authorisation for all testing described in this report.

3. Methodology

The work followed a standard, enumeration-led cloud penetration-testing lifecycle:

1. Pre-engagement / Scoping - confirmed the authorised target and objective of each lab before interacting with it.
2. Identity enumeration - confirmed the active principal and mapped its permissions using aws sts get-caller-identity and the aws iam list-* commands.
3. Resource enumeration - tested access to storage resources, including anonymous / unsigned access using aws s3 ls and the --no-sign-request option.
4. Access and validation - retrieved accessible objects to confirm impact and captured the challenge flag as proof of completion.
5. Reporting - documented each finding with timestamped evidence, a risk rating, impact and remediation.

4. Tools Used

Tool	Purpose	Where used
AWS CLI v2	Identity and S3 enumeration; sts get-caller-identity, iam list-*, s3 ls / cp.	IAM lab, flaws.cloud
--no-sign-request	Anonymous (unsigned) S3 access to test for public exposure.	flaws.cloud Level 1
Web browser	Accessing lab portals and confirming exposed S3 objects over HTTP.	flaws.cloud, TryHackMe
Pwned Labs / flaws.cloud / TryHackMe	Authorised training platforms hosting the target environments.	All activities

5. Lab Summaries

5.1 Intro to AWS IAM Enumeration (Pwned Labs) - Completed

Link: <https://pwnedlabs.io> | Status: Completed (flag accepted)

Scenario: acting as a security consultant for the fictional "Huge Logistics" following suspicious activity, the task was to enumerate IAM user dev01 and map any potentially compromised resources.

What was done: Using a set of leaked dev01 access keys, the active identity was confirmed and the user's group and policy attachments were enumerated. dev01 was found to carry AmazonGuardDutyReadOnlyAccess plus a custom dev01 policy, and to hold multiple access keys. Enumeration revealed an accessible internal S3 bucket, hl-dev-artifacts, containing build artifacts and a sensitive flag.txt, which was read directly to confirm the exposure and complete the lab.

Security issue understood: Long-lived access keys combined with excessive permissions let an attacker move from stolen credentials to reading sensitive internal data with no exploit required.

Defensive lesson: Apply least privilege, replace long-lived keys with short-lived role-based credentials, rotate and remove unused keys, and monitor API activity with CloudTrail and GuardDuty.

5.2 flaws.cloud - Level 1 (Completed)

Link: <http://flaws.cloud/> | Status: Level 1 solved

Scenario: flaws.cloud is a static website hosted directly from an S3 bucket whose name matches the domain. Level 1 demonstrates how a publicly listable bucket exposes files assumed to be private.

What was done: The bucket was listed anonymously using an unsigned request. The listing returned the site's normal files plus a hidden object, secret-dd02c7c.html, that is not linked anywhere on the site. Opening that object in a browser confirmed the level was solved and revealed the Level 2 URL.

Security issue understood: A bucket that allows anonymous s3:ListBucket lets anyone enumerate its contents; hiding a file behind an unguessable name ("security through obscurity") is not an access control.

Defensive lesson: Enable S3 Block Public Access, remove public/AllUsers grants, and protect objects with least-privilege policies rather than obscure names.

5.3 AWS Shared Responsibility Model (TryHackMe) - Completed 100%

Link: <https://tryhackme.com> | Status: Completed (100%)

This conceptual room clarified the boundary between what AWS secures ("security of the cloud" - hardware, global infrastructure, managed-service internals) and what the customer secures ("security in the cloud" - IAM, data, configuration and network rules). It reinforced the labs above through real-world case studies including Capital One (SSRF to an over-permissioned role reading S3), Uber (exposed credentials) and the recurring "it was just a bucket" public-S3 pattern. Completion is shown in Figure 4.

6. Findings

Both findings below were observed by me on authorised training targets, and each is supported by a screenshot in Appendix A.

Finding CLOUD-01 - Over-permissioned IAM user with exposed access keys

Finding ID	CLOUD-01
Vulnerability / Security Issue	Over-permissioned IAM user with exposed / long-lived access keys
Severity	High
Affected Lab / Target	Pwned Labs - Intro to AWS IAM Enumeration; IAM user dev01 (AWS account 794929857501)
Description	The dev01 user's long-lived access keys were available to an attacker. Once authenticated as dev01, IAM enumeration was possible with no restriction, exposing the user's attached policies and access-key tags. The user's effective permissions were broad enough to discover and read an internal build-artifacts bucket (hl-dev-artifacts) containing sensitive content, retrieved as flag.txt.
Evidence	Figure 1
Steps Performed	1. Confirmed identity with <code>aws sts get-caller-identity (user/dev01)</code>. 2. Enumerated group and policy attachments with <code>aws iam list-groups-for-user</code> and <code>list-attached-user-policies</code>. 3. Listed the internal bucket: <code>aws s3 ls s3://hl-dev-artifacts --recursive</code>. 4. Read the sensitive object: <code>aws s3 cp s3://hl-dev-artifacts/flag.txt - (flag d8ae4495888545df0c904551935c7514)</code>.
Impact	Read access to internal artifacts can expose source code, secrets and further credentials, enabling lateral movement and privilege escalation. Leaked long-lived keys are routinely harvested from code and config, making this highly likely to be exploited.
Recommendation	Apply least privilege to dev01; replace long-lived access keys with short-lived role-based credentials (<code>sts:AssumeRole</code>) and SSO; enforce key rotation and remove unused keys; require MFA; and monitor with CloudTrail and GuardDuty.

Finding CLOUD-02 - Publicly listable S3 bucket exposes hidden object

Finding ID	CLOUD-02
Vulnerability / Security Issue	Publicly listable S3 bucket; security through obscurity
Severity	Medium
Affected Lab / Target	flaws.cloud (Level 1); bucket flaws.cloud, region us-west-2
Description	The flaws.cloud bucket permitted <code>s3:ListBucket</code> to everyone, including unauthenticated (anonymous) callers. An unsigned request returned the full object listing, including a hidden file, <code>secret-dd02c7c.html</code> , that was not linked from the website. Because the file was protected only by an unguessable name rather than by a

	policy, anyone able to list the bucket could discover and open it.
Evidence	Figures 2 and 3
Steps Performed	1. Listed the bucket anonymously: <code>aws s3 ls s3://flaws.cloud/ --no-sign-request --region us-west-2</code>. 2. Identified the hidden object <code>secret-dd02c7c.html</code> in the listing. 3. Opened <code>http://flaws.cloud/secret-dd02c7c.html</code>, confirming Level 1 solved and revealing the Level 2 URL.
Impact	Exposure of any file placed in the bucket, including files assumed to be private. Public buckets are continuously scanned by automated tools, so exposure is likely; impact escalates if secrets are stored in the bucket.
Recommendation	Enable S3 Block Public Access at the account and bucket level; remove AllUsers / AuthenticatedUsers grants and public bucket-policy statements; never rely on unguessable object names for confidentiality; and enable encryption and access logging.

7. Risk Rating and Impact Analysis

Finding	Severity	Justification (likelihood x impact)
CLOUD-01	High	Very high likelihood - leaked long-lived keys are routinely harvested. High impact - read access to internal artifacts enables lateral movement and privilege escalation.
CLOUD-02	Medium	High likelihood - public buckets are continuously scanned. Medium impact - exposes any object in the bucket; escalates if secrets are stored there.

Overall risk: both findings sit on the customer side of the AWS Shared Responsibility Model. AWS secured the underlying platform; the exposures came entirely from customer-controlled identity permissions and bucket configuration - the same class of error behind the Capital One and "just a bucket" breaches.

8. Recommendations

- Apply least privilege to every IAM identity; scope users and roles to only the actions and resources they need. (Addresses CLOUD-01)
- Replace long-lived access keys with short-lived, role-based credentials and SSO; enforce rotation and remove unused keys. (Addresses CLOUD-01)
- Enable S3 Block Public Access at account and bucket level, and remove public/AllUsers grants. (Addresses CLOUD-02)
- Never rely on unguessable names for confidentiality; protect objects with policies and encryption. (Addresses CLOUD-02)
- Require MFA on all users and use permission boundaries or SCPs to cap what any single identity can do. (General)

- Enable CloudTrail, GuardDuty and IAM Access Analyzer to detect anomalous enumeration, public exposure and credential misuse. (General)

10. Conclusion

The objective of this assignment was to complete at least two free cloud-security labs (one IAM-focused and one storage/cloud-risk) plus a TryHackMe cloud room, and to document the learning, evidence and findings as a professional report. All three activities were completed, and two findings were produced, rated Medium and High.

The clearest technical takeaway was how directly a low-effort weakness leads to data exposure: leaked, over-permissioned IAM keys gave read access to an internal S3 bucket (CLOUD-01), and a publicly listable bucket exposed a file assumed to be private (CLOUD-02). Neither required any exploit. In every case AWS behaved correctly - the weaknesses were customer-side configuration choices, reinforcing the central lesson of the Shared Responsibility Model. The reporting phase reinforced that finding an issue is only half the work; expressing it with accurate evidence, a defensible risk rating and actionable remediation is what makes it useful.

Appendix A - Evidence / Screenshots

```
Command Prompt

{"UserId": "AIDA35FMDAPQWRTBSCOE",
 "Arn": "arn:aws:iam:794929857501:user/dev01",
 "CreateDate": "2023-09-28T21:56:31+00:00",
 "PasswordLastUsed": "2026-09-28T15:17:11+00:00",
 "Tags": [
  {
    "Key": "AKIA35FMDAPQWRTBSCOE",
    "Value": "dev01"
  },
  {
    "Key": "AKIA35FMDAPQWRTBSCOE",
    "Value": "dev01"
  }
]
}

C:\Users\geniu\Downloads>aws iam list-groups-for-user --user-name dev01
{
  "Groups": []
}

C:\Users\geniu\Downloads>aws iam list-attached-user-policies --user-name dev01
{
  "AttachedPolicies": [
    {
      "PolicyName": "AmazonGuardDutyReadOnlyAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AWSGuardDutyReadOnlyAccess"
    },
    {
      "PolicyName": "dev01",
      "PolicyArn": "arn:aws:iam::794929857501:policy/dev01"
    }
  ]
}

C:\Users\geniu\Downloads>aws s3 ls s3://hl-dev-artifacts --recursive
2023-10-01 23:39:53      1235 android-hotlin-extensions-tooling-232.9921.47.pom
2023-10-01 23:39:53     214036 android-project-system-gradle-models-232.9921.47-sources.jar
2023-10-01 23:38:05         32 flag.txt

C:\Users\geniu\Downloads>2023-10-01 23:38:05      32 flag.txt
'2023-10-01' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\geniu\Downloads>aws s3 cp s3://hl-dev-artifacts/flag.txt -
dbae4495888545df0c9045f
C:\Users\geniu\Downloads>
```

Figure 1 - Intro to AWS IAM Enumeration: enumeration of IAM user dev01 (identity, groups, attached policies) and direct read of flag.txt from the internal hl-dev-artifacts S3 bucket, yielding the accepted completion flag.

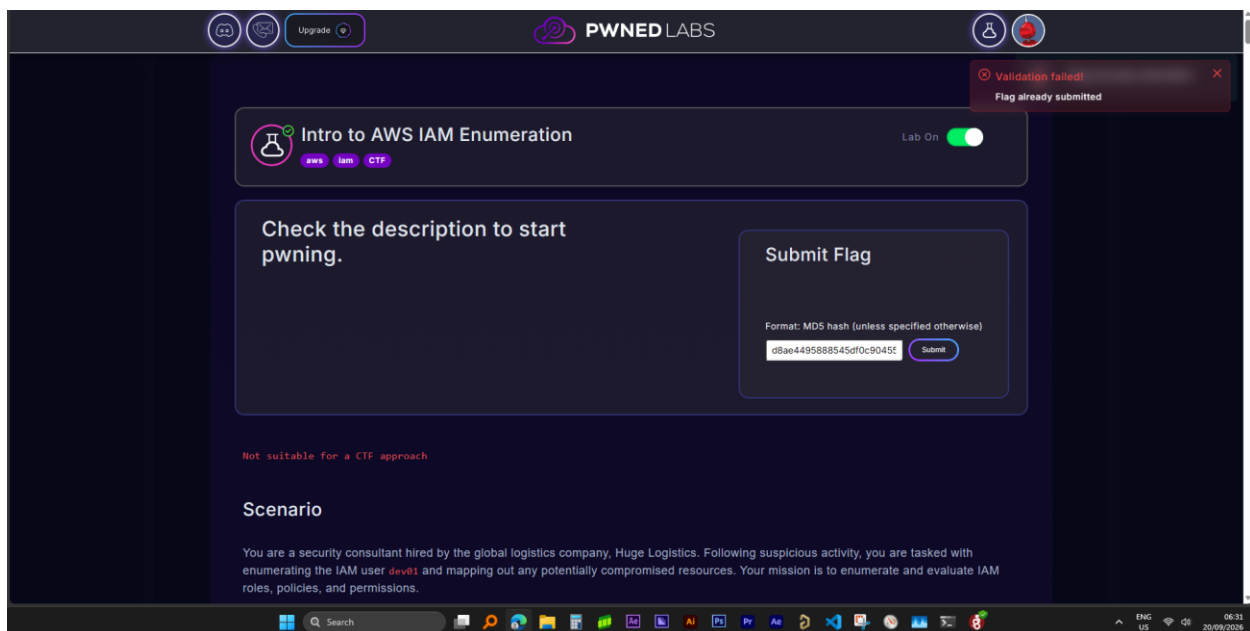


Figure 2 - Intro to AWS IAM Enumeration: flag captured.

```
Administrator: C:\Windows\system32\cmd.exe
C:\Users\geniu\Downloads\Professional documents\CyberSec Portfolio\AstraQuantum\WEEK6>aws s3 ls s3://flaws.cloud/ --no-sign-request --region us-west-2
2017-03-14 06:00:38      2575 hint1.html
2017-03-03 07:05:17      1707 hint2.html
2017-03-03 07:05:11      1101 hint3.html
2024-02-22 05:32:41      2861 index.html
2018-07-10 19:47:16    15979 logo.png
2017-02-27 04:59:28         46 robots.txt
2017-02-27 04:59:30      1051 secret-dd02c7c.html

C:\Users\geniu\Downloads\Professional documents\CyberSec Portfolio\AstraQuantum\WEEK6>
```

Figure 3 - flaws.cloud Level 1: anonymous (--no-sign-request) listing of the flaws.cloud S3 bucket, exposing the hidden object secret-dd02c7c.html.

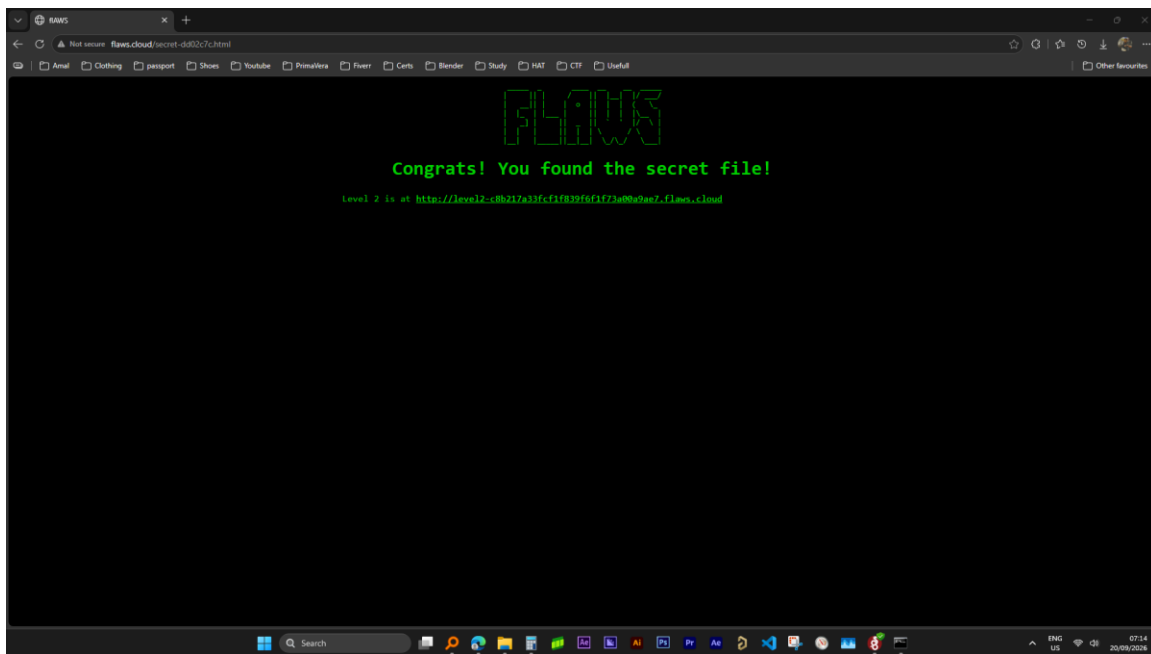


Figure 4 - flaws.cloud Level 1: the exposed hidden object opened in a browser, confirming the level was solved and revealing the Level 2 URL.

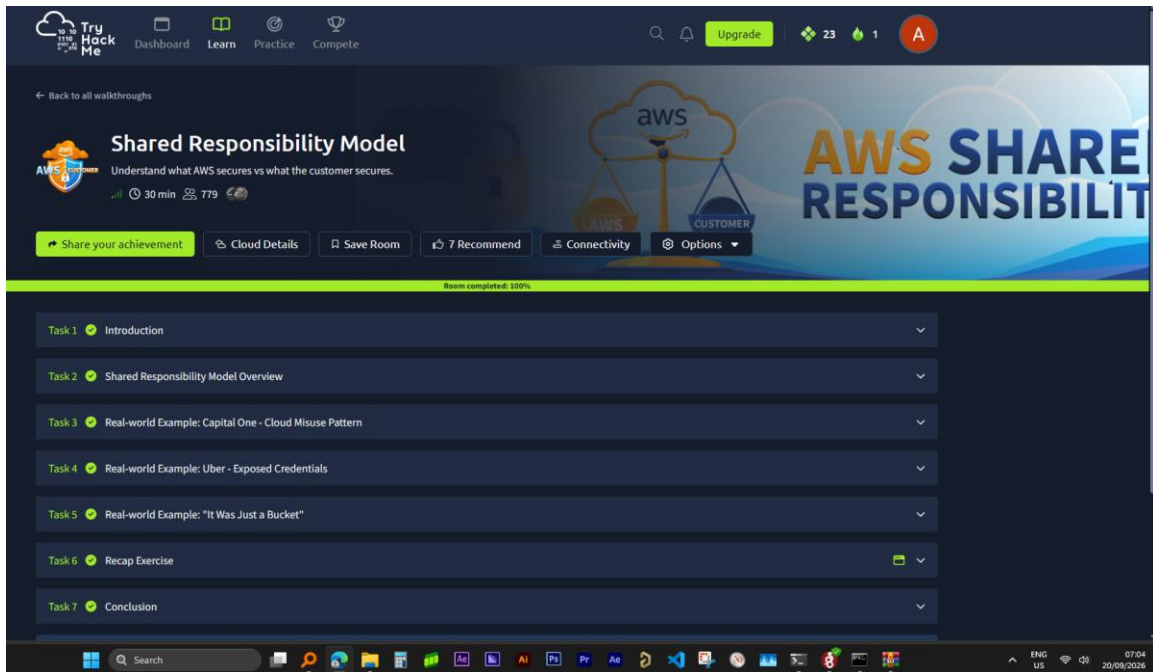


Figure 5 - TryHackMe "AWS Shared Responsibility Model" room shown at 100% completion.